CSE 209 — Computer Architecture

ALU Design · Arithmetic, Logic and Status Circuits (Mano ch. 9)

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Tushar Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — ALU questions and how much they matter

Lesson 5 — Adders and the ALU Block — half adder, full adder, parallel adder, ALU inputs and selection lines

Lesson 6 — The Arithmetic Circuit: Function Table and Output Carry — B-input logic, Table 9-1, when Cout = 1

Lesson 7 — Designing an Arithmetic Unit: the X/Y/Z Method — identities, recipe, five TF designs solved and verified

Lesson 8 — Logic Circuit and the Complete ALU — mux design, efficient design, Table 9-4, one typical stage

Lesson 9 — Status Register, Overflow and Comparator Circuits — C, S, Z, V; why V = XOR of last carries; comparison circuits

Quick Notes — ALU Design — merged revision notes + key diagrams

> **NOTE** Slide numbers refer to ALU Design.pdf (Tushar Sir, 23 slides, from Mano, Digital Logic and Computer Design §9.3–9.7). Every ALU design in these notes was checked by simulating all 256 input pairs (A, B) of a 4-bit ALU for every selection code (script _build/verify.py).

Exam Map — What the TF Papers Ask

Every Tushar-Sir-part question that this deck answers (★ = 2019-20 or later):

Session (paper)	Q	What is asked	Marks	Lesson
2023-24 (CSE 209) ★	5(b)	4-bit ALU: Xi, Yi, Zi for A−B−1, A+B+1, transfer A, decrement A, A OR B, A'	15	L7
2022-23 (CSE 209) ★	11(a)	Why is the overflow bit V computed by XOR-ing the last two carry bits?	7	L9
2022-23 (CSE 209) ★	11(d)	4-bit ALU Xi, Yi, Zi for −A−B−2, B + A' (1's comp of A to B), A+B, A−B, AND, XOR	10	L7
2022-23 (CSE 305) ★	5(b)	4-bit ALU with parallel SUBTRACTORS only: A−B, A+B, −A−B−2, B−A, XOR, AND	14	L7
2020-21 (CSE 305) ★	10(b)	4-bit ALU (S3–S0), 12 functions incl. A−1, A+B+1, A−B−1, A', AB', XNOR, OR: Xi, Yi, Zi with least gates	18	L7
2020-21 (CSE 305) ★	12(d)	Status module (V Z S C of A−B): circuit that decides A > B, A = B, A < B	8	L9
2019-20 (CSE 305) ★	2(a)	4-bit ALU (S3–S0): A', A'+1, A'+B, A'+B+1, A'−B−1, A'−B, A'−1, NAND, XNOR, XOR, OR — Xi, Yi, Zi, least gates	20	L7
2019-20 (CSE 305) ★	2(b)	Circuit with status-generator modules + gates for F = ((A>=B && C!=D) || X==0)	15	L9
2018-19 (CSE 305)	5(d)	4-bit ALU: add, transfer A, A+B', increment A, XOR, NOT, AND, OR — one typical stage	10	L8
2017-18 (CSE 305)	6(a)	4-bit ALU: add, transfer A, decrement A, A+B', XOR, AND, NOT, OR — one typical stage	25	L8
2017-18 (CSE 305)	6(b)	For each arithmetic operation of 6(a): when is the output carry 1?	10	L6

EXAM FOCUS The ALU design question — 'derive X_i, Y_i, Z_i for the parallel adders of an ALU that performs this table' — is asked in **every paper from 2017-18 to 2023-24** (7 times, 10–25 marks each; 107 marks since 2019-20). It is the single most valuable topic of the ART part. Master Lesson 7's recipe and its identity table, then practise the five solved designs cold. Lesson 9 (status bits) gives a further 8–15 marks most years.

Lesson 5 — Adders and the ALU Block

Slides 1–5 (ALU Design)

1. What an ALU Is (sl. 2)

An Arithmetic Logic Unit performs both arithmetic and logic micro-operations on two n-bit inputs A and B and produces F.

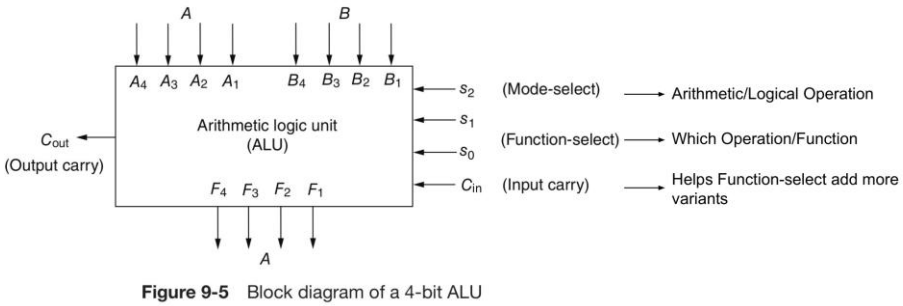


Figure 9-5 Block diagram of a 4-bit ALU

Figure 5.1 — Block diagram of a 4-bit ALU (sl. 2)

Signal	Name	Job
s2	Mode select	Chooses arithmetic (s2 = 0) or logic (s2 = 1) operation
s1 s0	Function select	Chooses which operation inside the mode
C_{in}	Input carry	Adds 1 to arithmetic results — doubles the number of arithmetic variants
C_{out}	Output carry	Carry out of the most significant stage (used by the status register)

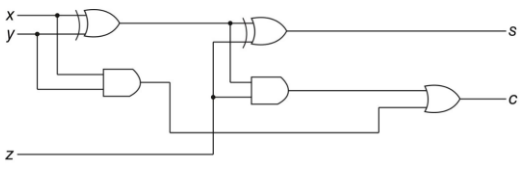
2. Half Adder and Full Adder (sl. 4–5)

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Half adder: $S = x'y + xy' = x \oplus y, \quad C = xy$

Full adder: $S = x'y'z + x'yz' + xy'z' + xyz = x \oplus y \oplus z, \quad C = xy + xz + yz$

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$S = x'y'z + x'yz' + xy'z' + xyz$$
$$C = xy + xz + yz$$

Figure 5.2 — Full adder: truth table, equations and gate circuit (two half adders + OR) (sl. 5)

3. Parallel Adder (sl. 3)

An n-bit parallel adder is n full adders in cascade: the carry out of stage i is the carry in of stage i + 1. C_1 is the input carry, C_{n+1} the output carry.

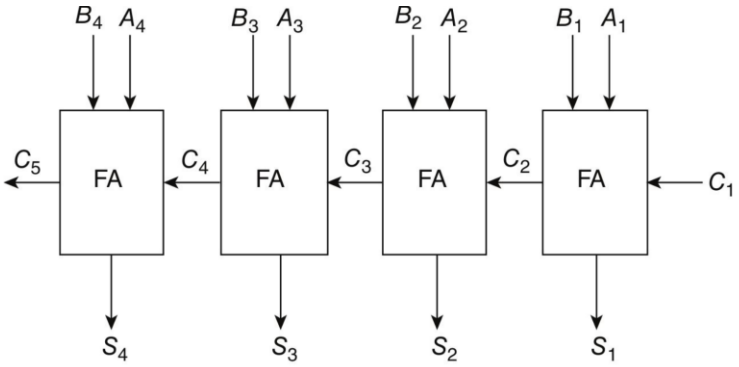


Figure 5.3 — 4-bit parallel adder built from full adders (sl. 3)

MEMORIZE The full adder of stage i always computes $F_i = X_i \oplus Y_i \oplus Z_i$ and $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$. An ALU is built by putting small combinational circuits in front of this adder that decide what X_i, Y_i and the carry Z_i are.

Practice — Lesson 5

Constructed (adder basics).

(a) Add A = 0111 and B = 0110 on a 4-bit parallel adder with $C_1 = 0$. Give every carry and the result.

(b) Show that a full adder can be built from two half adders and an OR gate.

✓ Solution

(a) Bit 1 is the LSB; $C_1 = 0$.

```
Stage     1    2    3    4
A_i       1    1    1    0       (A = 0111, bit 1 = LSB)
B_i       0    1    1    0       (B = 0110)
C_i       0    0    1    1
F_i       1    0    1    1  → F = 1101,  C_5 = 0
Unsigned: 7 + 6 = 13 = 1101 ✓.  Signed 4-bit: +7 + 6 = +13 → overflow (result looks −3)
```

(b) HA1 adds x and y: S1 = x ⊕ y, C1 = xy. HA2 adds S1 and z: S = x ⊕ y ⊕ z, C2 = (x ⊕ y)z. OR: C = xy + (x ⊕ y)z = xy + xz + yz — exactly the circuit on slide 5.

Lesson 6 — The Arithmetic Circuit: Function Table and Output Carry

Slides 6–11 (ALU Design)

1. Eight Operations From One Parallel Adder (sl. 6–7)

Keep A on the X inputs and change only what goes into the Y inputs (B, B', all 0s or all 1s) and the input carry:

Y input	$C_{in} = 0$	$C_{in} = 1$
B	F = A + B (addition)	F = A + B + 1 (add with carry)
B' (1's complement)	F = A + B' (A plus 1's complement of B = A − B − 1)	F = A + B' + 1 = A − B (subtraction)
All 0s	F = A (transfer A)	F = A + 1 (increment)
All 1s (= −1)	F = A − 1 (decrement)	F = A − 1 + 1 = A (transfer A)

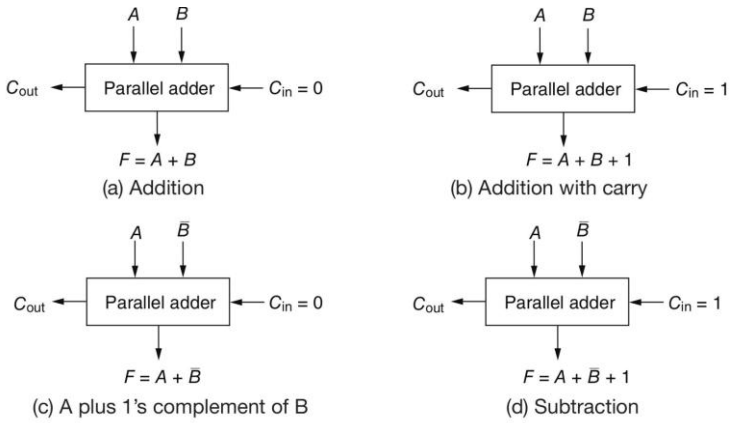
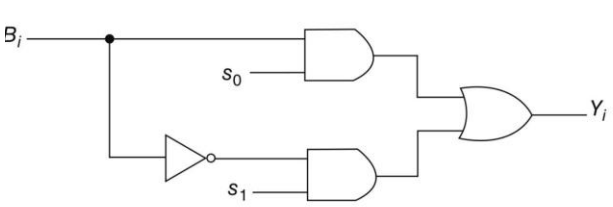


Figure 6.1 — Addition, add with carry, A plus 1's complement of B, subtraction (sl. 6)

2. True/Complement, One/Zero Circuit (sl. 8)

One gate pair per bit chooses what reaches Y_i:

$X_i = A_i$ $\quad Y_i = B_i s_0 + B_i' s_1$ $\quad (s_1 s_0$: 00 → 0, 01 → B_i, 10 → B_i', 11 → 1)



s_1	s_0	Y_i
0	0	0
0	1	B_i
1	0	B'_i
1	1	1

Figure 9-7 True/complement, one/zero circuit

$$X_i = A_i$$
$$Y_i = B_i s_0 + B'_i s_i \qquad i = 1, 2, \ldots n$$

Figure 6.2 — True/complement, one/zero circuit (Fig. 9-7) (sl. 8)

3. The Complete Arithmetic Circuit and Its Function Table (sl. 9–10)

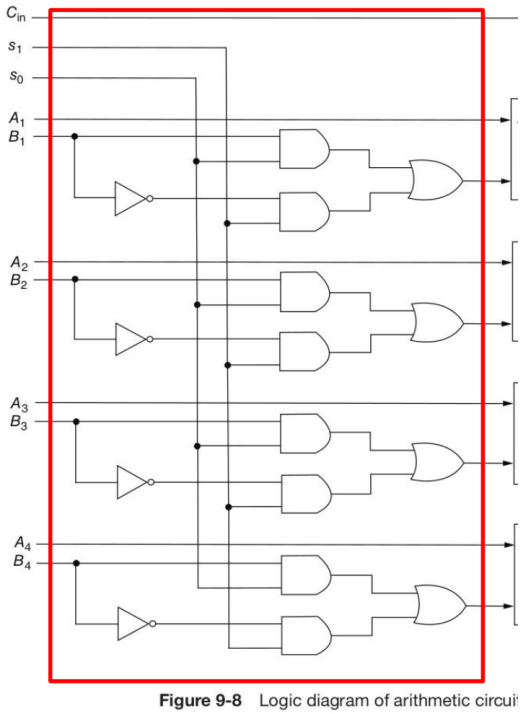


Figure 6.3 — 4-bit arithmetic circuit (Fig. 9-8): B-input logic (red box) in front of four full adders (sl. 9)

s1	s0	C_{in}	Y equals	Output	Function
0	0	0	0	F = A	Transfer A
0	0	1	0	F = A + 1	Increment A
0	1	0	B	F = A + B	Add B to A
0	1	1	B	F = A + B + 1	Add B to A plus 1
1	0	0	B'	F = A + B'	Add 1's complement of B to A
1	0	1	B'	F = A + B' + 1	Add 2's complement of B to A (A − B)
1	1	0	All 1s	F = A − 1	Decrement A
1	1	1	All 1s	F = A	Transfer A

Table 9-1 (sl. 10). The generic picture (sl. 9, right): selection variables + A + B → combinational circuit → X, Y → n-bit parallel adder (with C_{in}). Every design question in Lesson 7 only redesigns that combinational circuit.

4. When Is the Output Carry 1? (sl. 11) ★

s1 s0 C_{in}	Function	C_{out} = 1 if	Comment (n-bit unsigned)
000	F = A	never	C_{out} is always 0
001	F = A + 1	A = 2^n − 1	C_{out} = 1 and F = 0 if A = 2^n − 1 (all 1s)
010	F = A + B	A + B ≥ 2^n	overflow (unsigned) when C_{out} = 1
011	F = A + B + 1	A + B ≥ 2^n − 1	overflow when C_{out} = 1
100	F = A − B − 1	A > B	if C_{out} = 0 then A ≤ B and F = 1's complement of (B − A)
101	F = A − B	A ≥ B	if C_{out} = 0 then A < B and F = 2's complement of (B − A)
110	F = A − 1	A ≠ 0	C_{out} = 1 except when A = 0
111	F = A	always	C_{out} is always 1

Reason: the adder really computes A + Y + C_{in} on n bits. For A − B − 1 = A + B' = A + (2^n − 1 − B): the sum reaches 2^n exactly when A − B − 1 ≥ 0, i.e. A > B. For A − 1 = A + (2^n − 1): the sum reaches 2^n when A ≥ 1.

TRAP In subtraction a carry of 1 means **no borrow** (A ≥ B). Do not read C_{out} = 1 as 'overflow' for the subtract rows.

Exam Approach

1. Rewrite each operation as A + Y + C_{in} with Y ∈ {B, B', 0, 1s}.
2. For carry questions compute when the true sum A + Y + C_{in} ≥ 2^n and translate back (A ≥ B, A ≠ 0 …).
3. Quote Table 9-1 / 9-2 rows exactly; the examiner checks the conditions.

Practice — Lesson 6

(a) Actual — 2017-18 6(b). (b) Constructed.

(a) For each arithmetic operation of 2017-18 6(a) — Addition (A + B), Transfer A, Decrement A, Add 1's complement of B to A — find the condition under which the output carry is 1. (10)

(b) A = 0101, B = 0011. Compute A − B on the arithmetic circuit (s1 s0 C_{in} = 101) and B − A (swap inputs). Interpret C_{out} each time.

✓ Solution

Operation	Adder computes	C_{out} = 1 when (4-bit)	Example
Addition	A + B	A + B ≥ 16 (unsigned overflow)	1001 + 1000: C_{out} = 1
Transfer A	A + 0 + 0	never — C_{out} = 0 always	—
Decrement A	A + 1111	A ≠ 0	0000 + 1111 = 1111, C_{out} = 0
A + B'	A + (15 − B)	A > B	0101 + 1100 = 1 0001: C_{out} = 1 (5 > 3)

```
A − B: 0101 + 1100 + 1 = 1 0010 → F = 0010 (= 2), Cout = 1 → A ≥ B, result is A − B
B − A: 0011 + 1010 + 1 = 0 1110 → F = 1110, Cout = 0 → B < A, F = 2's complement of 2 (−2)
```

Lesson 7 — Designing an Arithmetic Unit: the X/Y/Z Method

Slides 12–14 (ALU Design) + method used by every TF design question

1. The Slide Design Example: Adder/Subtractor (sl. 12–14)

Design: one selection variable s; s = 0 → F = A + B; s = 1 → F = A − B by taking the 2's complement of B.

s	A_i	B_i	X_i	Y_i
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	0
1	1	0	1	1
1	1	1	1	0

$$X_i = A_i \qquad Y_i = B_i \oplus s \qquad C_{in} = s$$

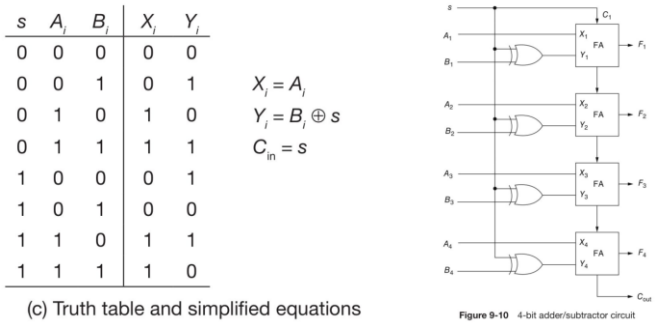


Figure 7.1 — Truth table, equations and the 4-bit adder/subtractor (Fig. 9-10) (sl. 14)

2. Identities You Need (n-bit 2's complement, A' = 1's complement)

Wanted F	Write it as X + Y + C_{in}	X	Y	C_{in}
A + B	A + B + 0	A	B	0
A + B + 1	A + B + 1	A	B	1
A − B	A + B' + 1	A	B'	1
A − B − 1	A + B' + 0	A	B'	0
A	A + 0 + 0 (or A + 1s + 1)	A	0 (1s)	0 (1)
A + 1	A + 0 + 1	A	0	1
A − 1	A + 1111 + 0	A	1s	0
B − A	A' + B + 1	A'	B	1
A' (+ ...)	A' + Y + C	A'	as needed	as needed
−A − B − 2	A' + B' (−A = A' + 1, −B = B' + 1)	A'	B'	0
B + A' (add 1's comp. of A to B)	A' + B + 0	A'	B	0

Logic functions from the adder (all carries forced to 0, so $F_i = X_i \oplus Y_i$)

Wanted F_i	X_i	Y_i	Check
A (transfer)	A	0	$A \oplus 0 = A$
A' (NOT)	A	1	$A \oplus 1 = A'$
$A \oplus B$ (XOR)	A	B	—
$A \odot B$ (XNOR)	A	B'	$A \oplus B' = (A \oplus B)'$
A + B (OR)	A + B	0	OR the B bit into X
AB (AND)	A + B'	B'	$B = 0 \rightarrow 1 \oplus 1 = 0$; $B = 1 \rightarrow A \oplus 0 = A$
AB'	A + B	B	$B = 1 \rightarrow 1 \oplus 1 = 0$; $B = 0 \rightarrow A$
(AB)' (NAND)	A' + B'	0	De Morgan
A + B via A' (OR)	A'B'	1	$(A'B')' = A + B$

MEMORIZE Carry masking: the carry into every stage is $Z_i = s_2' C_i$ (mode bit = 0 for arithmetic). In logic mode $s_2 = 1$ kills all carries, so each full adder becomes an XOR gate: $F_i = X_i \oplus Y_i$. The input carry C_1 is a function of the selection lines only.

3. The Recipe (use it for every TF design)

1. For every arithmetic row write F as $X + Y + C_{in}$; choose $X \in \{A, A'\}$, $Y \in \{0, B, B', 1s\}$. Where two choices exist (transfer A), pick the one that makes the equations simpler.
2. For every logic row choose X_i, Y_i from the logic table so that $X_i \oplus Y_i$ is the wanted function. Try to reuse the Y column of the arithmetic rows.
3. Make the design table: selection bits → X_i, Y_i, C_{in}. Treat unused selection codes and the 'x' bits as don't-cares.
4. Read off the equations (Karnaugh map or inspection on B_i and the s bits). Look for XOR forms such as $B_i \oplus s_0$.
5. Write $Z_i = s_{mode}' C_i$ and C_1 = (expression of the s bits). Draw one typical stage: X-logic, Y-logic, AND gate for Z_i, full adder.
6. Check two rows by substitution (one arithmetic, one logic) in the script.

4. ★ Worked TF Designs (all verified by simulation)

A. 2023-24 5(b) (15 marks)

S2 S1 S0: 000 A − B − 1 · 001 A + B + 1 · 010 transfer A · 011 decrement A · 10x A OR B · 11x A'.

S2 S1 S0	Function	X_i	Y_i	C_{in}
000	A − B − 1 = A + B'	A	B'	0
001	A + B + 1	A	B	1
010	Transfer A = A + 1s + 1	A	1	1
011	A − 1 = A + 1s	A	1	0
10x	A OR B	A + B	0	— (carries 0)
11x	A'	A	1	— (carries 0)

Arithmetic Y: S1 = 0 → B' (S0 = 0) or B (S0 = 1), i.e. $B_i \odot S0$; S1 = 1 → 1. Choosing 'A + 1s + 1' for transfer A makes the S1 = 1 rows both Y = 1.

$$X_i = A_i + S2 \cdot S1' \cdot B_i$$
$$Y_i = S1 + S2' \cdot (B_i \odot S0)$$

$$C_{in} = S2'\cdot(S1 \oplus S0) \qquad Z_i = S2'\cdot C_i$$

Check: 011 → Y = 1, C_{in} = 0 → A + 1111 = A − 1 ✓. 11x → X = A, Y = 1, carries 0 → F = A ⊕ 1 = A' ✓.

B. 2022-23 11(d) (10 marks)

S2 S1 S0: 000 −A − B − 2 · 001 add 1's complement of A to B · 010 A + B · 011 A − B · 10x A AND B · 11x A XOR B.

S2 S1 S0	Function	X_i	Y_i	C_{in}
000	−A − B − 2 = A' + B'	A'	B'	0
001	B + A'	A'	B	0
010	A + B	A	B	0
011	A − B = A + B' + 1	A	B'	1
10x	A AND B	A + B'	B'	—
11x	A XOR B	A	B	—

$$X_i = (A_i \oplus S2'S1') + S2\cdot S1'\cdot B_i'$$
$$Y_i = B_i \odot (S1 \oplus S2'S0)$$
$$C_{in} = S2'\cdot S1\cdot S0 \qquad Z_i = S2'\cdot C_i$$

Why: arithmetic X is A' exactly when S1 = 0 (A ⊕ S1'); Y is B when S1 ⊕ S0 = 1, else B'. In logic mode S0 is a don't-care, so the term S2'S0 removes it: Y = B (S1 = 1) or B' (S1 = 0) as required.

C. 2022-23 (CSE 305) 5(b) — parallel SUBTRACTORS only (14 marks)

A full subtractor gives $D_i = X_i \oplus Y_i \oplus Z_i$ and borrow $B_{i+1} = X_i'Y_i + X_i'Z_i + Y_iZ_i$; the unit computes **F = X − Y − B_{in}**. So rewrite every operation as X − Y − borrow, using −B' = B + 1 (because B' = −B − 1).

S2 S1 S0	Function	As X − Y − B_{in}	X_i	Y_i	B_{in}
000	A − B	A − B − 0	A	B	0
001	A + B	A − B' − 1 = A + B + 1 − 1	A	B'	1
010	−A − B − 2	A' − B − 1 = (−A − 1) − B − 1	A'	B	1
011	B − A	A' − B' = (−A − 1) − (−B − 1)	A'	B'	0
100	A XOR B	borrows 0: D = X ⊕ Y	A	B	—
110	A AND B	borrows 0	A + B'	B'	—

$$X_i = (A_i \oplus S2'S1) + S2\cdot S1\cdot B_i'$$
$$Y_i = B_i \oplus (S0 + S2\cdot S1)$$
$$B_{in} = S2'\cdot(S1 \oplus S0) \qquad Z_i \text{ (borrow into stage i)} = S2'\cdot B_i$$

> **NOTE** With borrows forced to 0 a subtractor stage is also just X ⊕ Y, so the logic part is designed exactly as for adders. Codes 101 and 111 are unused (don't-cares).

D. 2020-21 10(b) — four selection lines, least gates (18 marks)

S3 S2 S1 S0	Function	X_i	Y_i	C_{in}
0000	A	A	0	0
0001	A − 1	A	1	0
0010	A + B + 1	A	B	1
0011	A + B	A	B	0
0100	A − B	A	B'	1

S3 S2 S1 S0	Function	X_i	Y_i	C_{in}
0101	A − B − 1	A	B'	0
0110	A + 1	A	0	1
0111	A (as A + 0 + 0)	A	0	0
100x	A'	A	1	—
101x	AB'	A + B	B	—
110x	A ⊙ B (XNOR)	A	B'	—
111x	A + B (OR)	A + B	0	—

Y depends only on S2 S1 (and S0 when S2S1 = 00): 00 → S0 in arithmetic, 1 in logic → (S0 + S3); 01 → B; 10 → B'; 11 → 0. X needs B only in rows 101x and 111x (S3 = 1, S1 = 1).

$$X_i = A_i + S3 \cdot S1 \cdot B_i$$
$$Y_i = S2'S1'(S0 + S3) + S2'S1 \cdot B_i + S2S1' \cdot B_i'$$
$$C_{in} = S3' \cdot S0' \cdot (S2 + S1) \qquad Z_i = S3' \cdot C_i$$

E. 2019-20 2(a) — arithmetic on A' (20 marks)

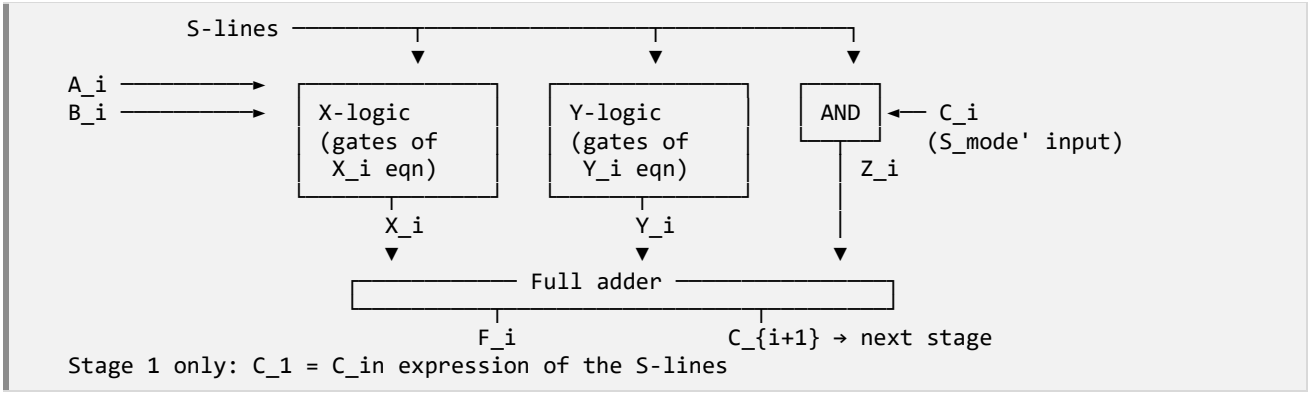
S3 S2 S1 S0	Function	X_i	Y_i	C_{in}
0000 / 0001	A' / A' + 1	A'	0	S0
0010 / 0011	A' + B / A' + B + 1	A'	B	S0
0100 / 0101	A' − B − 1 / A' − B	A'	B'	S0
0110 / 0111	A' − 1 / A'	A'	1	S0
100x	(AB)' NAND	A' + B'	0	—
101x	A ⊙ B XNOR	A'	B	—
110x	A ⊕ B XOR	A'	B'	—
111x	A + B OR	A'B'	1	—

This is Table 9-1 with A' in place of A and (S2, S1) in place of (s1, s0). The logic rows were chosen so that Y keeps the same pattern 0, B, B', 1 — so Y needs no S3 at all.

$$Y_i = S1 \cdot B_i + S2 \cdot B_i' \qquad \text{(same for both modes)}$$
$$X_i = A_i' \cdot (S3S2S1B_i)' + S3 \cdot S2' \cdot S1' \cdot B_i'$$
$$C_{in} = S3' \cdot S0 \qquad Z_i = S3' \cdot C_i$$

Check NAND (100x): X = A' + B', Y = 0 → F = A' + B' = (AB)' ✓. OR (111x): X = A'B', Y = 1 → F = (A'B')' = A + B ✓.

5. Drawing One Typical Stage



Exam Approach

1. Write the design table first (S bits | function | X | Y | C_{in}) — most marks are for it.
2. Reuse identities: −A = A' + 1, A − B = A + B' + 1, −A − B − 2 = A' + B', B − A = A' + B + 1.
3. Choose logic-row X, Y so that Y keeps the arithmetic pattern — fewer gates.
4. Give X_i, Y_i, C_{in}, Z_i; draw one stage; verify one row.
5. For subtractor-only designs use −B' = B + 1 and the borrow-in as C_{in}.

Practice — Lesson 7

(a) Actual — 2023-24 5(b). (b) Actual — 2022-23 11(d). (c) Actual — 2022-23 (CSE 305) 5(b). (d) Actual — 2020-21 10(b). (e) Actual — 2019-20 2(a).

(a) –(e) The five specifications are exactly those of §4 A–E. Cover the solutions and redo each one: design table → equations → one stage.

✓ Solution

(a)–(e) §4 A–E above. Each set of equations was simulated for all 4-bit A, B and every selection code — all outputs matched the required functions.

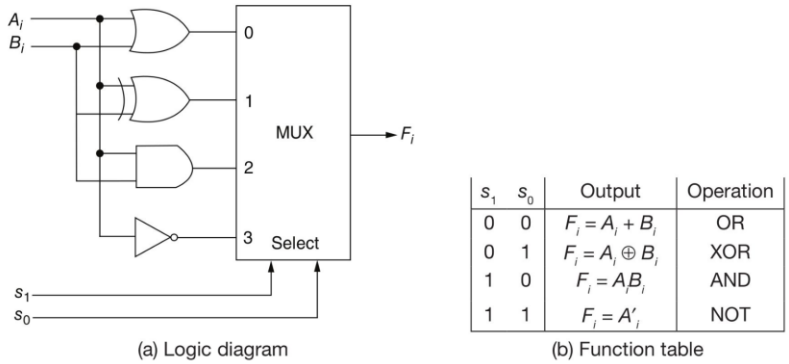
Lesson 8 — Logic Circuit and the Complete ALU

Slides 15–20 (ALU Design)

1. Logic Circuit With a Multiplexer (sl. 15)

The direct design: compute OR, XOR, AND and NOT of each bit pair and pick one with a 4-to-1 mux controlled by s1 s0.

s1 s0	Output	Operation
00	$F_i = A_i + B_i$	OR
01	$F_i = A_i \oplus B_i$	XOR
10	$F_i = A_iB_i$	AND
11	$F_i = A_i'$	NOT



s_1	s_0	Output	Operation
0	0	$F_i = A_i + B_i$	OR
0	1	$F_i = A_i \oplus B_i$	XOR
1	0	$F_i = A_iB_i$	AND
1	1	$F_i = A_i'$	NOT

Figure 9-11 One stage of logic circuit

Figure 8.1 — One stage of the logic circuit (Fig. 9-11) (sl. 15)

2. Combining Logic and Arithmetic (sl. 16)

One arithmetic stage and one logic stage, followed by a 2-to-1 mux controlled by the mode bit s2 (0 = arithmetic, 1 = logic).

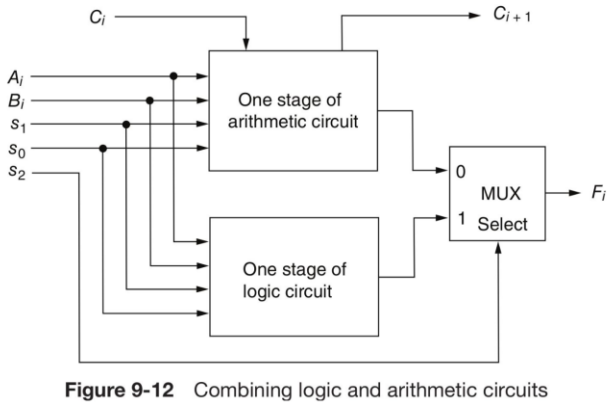


Figure 9-12 Combining logic and arithmetic circuits

Figure 8.2 — Combining logic and arithmetic circuits (Fig. 9-12) (sl. 16)

3. The More Efficient Design (sl. 17–20) ★

1. Design the arithmetic section independently of the logic section.

2. Find which logic operations the arithmetic circuit already produces when all input carries are 0.
3. Modify the arithmetic circuit to obtain the required logic operations.

With carries 0 each full adder gives $F_i = X_i \oplus Y_i$. With $X_i = A_i$ and the arithmetic Y_i (0, B, B', 1) that already produces (Table 9-3):

s2 s1 s0	X_i	Y_i	C_i	$F_i = X_i \oplus Y_i$	Operation	Required
1 0 0	A_i	0	0	A_i	Transfer A	OR
1 0 1	A_i	B_i	0	$A_i \oplus B_i$	XOR	XOR
1 1 0	A_i	B_i'	0	$A_i \odot B_i$	Equivalence	AND
1 1 1	A_i	1	0	A_i'	NOT	NOT

XOR and NOT are free. For OR, OR B_i into X_i in row 100 (X = A + B, Y = 0). For AND, OR B_i' into X_i in row 110 (X = A + B', Y = B' gives (A + B') $\oplus$ B' = AB). Carries are masked by s2:

$$X_i = A_i + s2 \cdot s1' \cdot s0' \cdot B_i + s2 \cdot s1 \cdot s0' \cdot B_i'$$
$$Y_i = s0 \cdot B_i + s1 \cdot B_i'$$
$$Z_i = s2' \cdot C_i$$

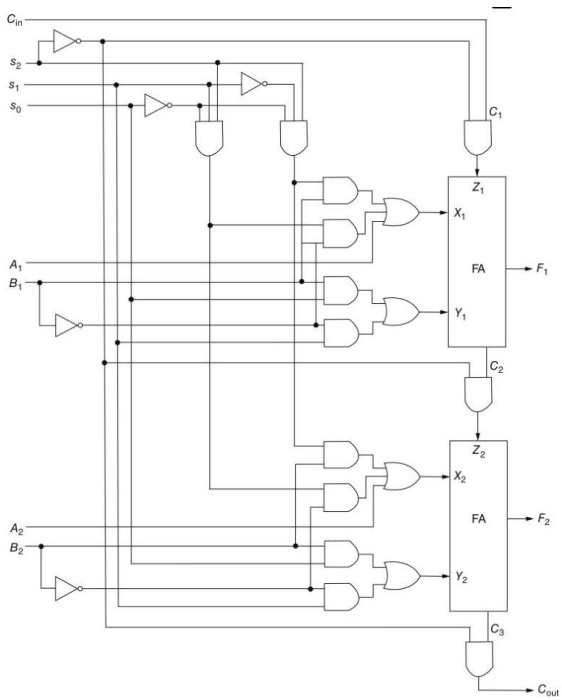


Figure 9-13 Logic diagram of arithmetic logic unit (ALU)

Figure 8.3 — Logic diagram of the ALU (Fig. 9-13), first two stages (sl. 19)

s2 s1 s0 C_{in}	Output	Function
0 0 0 0	F = A	Transfer A
0 0 0 1	F = A + 1	Increment A
0 0 1 0	F = A + B	Addition
0 0 1 1	F = A + B + 1	Add with carry
0 1 0 0	F = A − B − 1	Subtract with borrow
0 1 0 1	F = A − B	Subtraction
0 1 1 0	F = A − 1	Decrement A
0 1 1 1	F = A	Transfer A
1 0 0 X	F = A ∨ B	OR
1 0 1 X	F = A $\oplus$ B	XOR

s2 s1 s0 C_{in}	Output	Function
1 1 0 X	F = A ∧ B	AND
1 1 1 X	F = A'	Complement A

Table 9-4 (sl. 20) — the function table of the ALU of Fig. 9-13 (checked by simulation).

Exam Approach

1. 'Design an ALU … draw one typical stage': use the efficient method — arithmetic first, then get logic from X ⊕ Y with carries masked.
2. Without a C_{in} line in the question, C_{in} must come from the selection bits (here 0 for every arithmetic row, or a product term).
3. Draw: X-logic gates, Y-logic gates, AND gate (s2', C_i) → Z_i, full adder, F_i, C_{i+1}.

Practice — Lesson 8

(a) Actual — 2018-19 5(d). (b) Actual — 2017-18 6(a).

(a) 4-bit ALU, S2 S1 S0: 000 Addition, 001 Transfer A, 010 Add 1's complement of B to A, 011 Increment A, 100 XOR, 101 NOT, 110 AND, 111 OR. Draw one typical stage. (10)

(b) 4-bit ALU, S2 S1 S0: 000 Addition, 001 Transfer A, 010 Decrement A, 011 Add 1's complement of B to A, 100 XOR, 101 AND, 110 NOT, 111 OR. Draw one typical stage. (25)

✓ Solution

(a)

S2 S1 S0	Function	X_i	Y_i	C_{in}
000	A + B	A	B	0
001	Transfer A	A	0	0
010	A + B'	A	B'	0
011	A + 1	A	0	1
100	XOR	A	B	—
101	NOT	A	1	—
110	AND	A + B'	B'	—
111	OR	A + B	0	—

$$X_i = A_i + S2 \cdot S1 \cdot (S0 \odot B_i)$$
$$Y_i = S0' \cdot (B_i \oplus S1) + S2 \cdot S1' \cdot S0$$
$$C_{in} = S2' \cdot S1 \cdot S0 \qquad Z_i = S2' \cdot C_i$$

S0 ⊙ B_i gives B_i' for 110 (AND) and B_i for 111 (OR). Y is B, 0, B', 0 in arithmetic; logic needs B, 1, B', 0 — only row 101 differs, hence the extra term S2S1'S0.

(b) No input carry is listed, so C_{in} = 0 in every arithmetic row (A + B, A, A − 1 = A + 1s, A + B').

S2 S1 S0	Function	X_i	Y_i
000	A + B	A	B
001	Transfer A	A	0
010	Decrement A	A	1
011	A + B'	A	B'
100	XOR	A	B

S2 S1 S0	Function	X_i	Y_i
101	AND	A + B'	B'
110	NOT	A	1
111	OR	A + B	0

$$X_i = A_i + S2 \cdot S0 \cdot (S1 \odot B_i)$$
$$Y_i = S1'S0' \cdot B_i + S1 \cdot S0' + S0 \cdot B_i' \cdot (S2 \oplus S1)$$
$$C_{in} = 0 \qquad Z_i = S2' \cdot C_i$$

Y rows 00 and 10 are the same in both modes (B and 1); rows 01 and 11 swap between 0 and B' — the XOR (S2 ⊕ S1) selects B' for 011 and 101 only. One typical stage: Fig. in Lesson 7 §5 with these X/Y gates.

Lesson 9 — Status Register, Overflow and Comparator Circuits

Slides 21–22 (ALU Design)

1. The Four Status Bits (sl. 21–22)

Bit	Set when	Hardware (8-bit ALU)
C (carry)	Output carry of the ALU is 1	$C = C_9 = C_{out}$
S (sign)	Most significant bit of the result is 1	$S = F_8$
Z (zero)	Result is all 0s	$Z = (F_1 + F_2 + \dots + F_8)'$ — an 8-input NOR
V (overflow)	$C_8 \oplus C_9 = 1$: signed result > 127 or < −128	XOR of the carry into and out of the sign stage

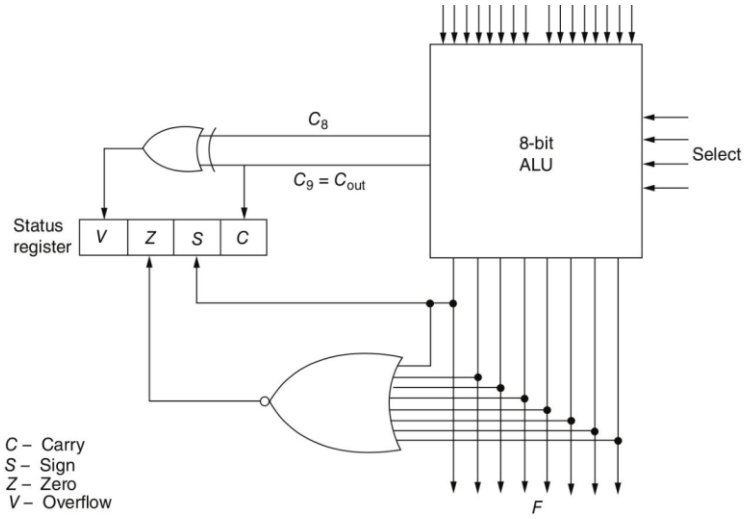


Figure 9-14 Setting bits in a status register

Figure 9.1 — Setting the bits of the status register (Fig. 9-14) (sl. 22)

2. Why V = XOR of the Last Two Carries ★

Let C_n be the carry into the sign stage and C_{n+1} the carry out of it (C_8, C_9 for 8 bits).

Operand signs	Carry out C_{n+1}	Sign of result	Overflow?
+ and + (MSBs 0, 0)	always 0	$= C_n$	yes exactly when $C_n = 1$ (result looks negative)
− and − (MSBs 1, 1)	always 1	$= C_n$	yes exactly when $C_n = 0$ (result looks positive)
+ and − (MSBs 0, 1)	$= C_n$	—	never (|sum| ≤ larger operand)

In both overflow cases the two carries differ, and whenever they are equal there is no overflow. Hence $V = C_n \oplus C_{n+1}$. Examples (4 bits): 0111 + 0001 = 1000 — carry into MSB 1, out 0 → V = 1 (+7 + 1 = +8 out of range). 1000 + 1111 = (1)0111 — into 0, out 1 → V = 1 (−8 − 1 = −9).

3. Comparing A and B With the Status Bits of A − B

The status module of the TF questions computes A − B = A + B' + 1 and outputs V, Z, S, C.

Relation	Unsigned A, B	Signed (2's complement) A, B
A = B	Z = 1	Z = 1

Relation	Unsigned A, B	Signed (2's complement) A, B
$A \neq B$	$Z = 0$	$Z = 0$
$A < B$	$C = 0$ (a borrow occurred)	$S \oplus V = 1$
$A \geq B$	$C = 1$	$S \oplus V = 0$
$A > B$	$C \cdot Z' = 1$	$(S \oplus V)' \cdot Z' = 1$
$A \leq B$	$C' + Z = 1$	$(S \oplus V) + Z = 1$

TRAP For signed numbers S alone is wrong: if A − B overflows the sign bit is flipped. That is why the test is $S \oplus V$. State whether you treat A, B as signed or unsigned.

Exam Approach

1. Decide signed or unsigned (TF status-module questions mean 2's complement unless they say otherwise; mention both if unsure).
2. Feed A and B to a module; take Z for equality and $S \oplus V$ (or C) for order.
3. Tests against 0: tie the second input to 0000 and use Z.
4. Combine with AND/OR/NOT exactly as the C condition says; draw modules as boxes with V Z S C outputs.

Practice — Lesson 9

(a) Actual — 2022-23 11(a). (b) Actual — 2020-21 12(d). (c) Actual — 2019-20 2(b).

(a) Why is the overflow bit (V) computed by XOR-ing the last two carry bits in binary arithmetic operations? (7)

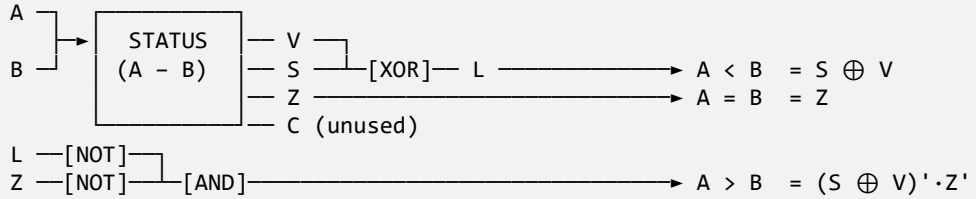
(b) A module takes two 4-bit inputs A and B and outputs only V, Z, S, C for A − B. Draw the complete circuit that decides whether A > B, A = B or A < B. (8)

(c) Implement F = 1 if ((A >= B && C != D) || X == 0) else 0 with basic gates and status-generator modules (4-bit A, B, C, D, X). (15)

✓ Solution

(a) §2: list the three sign cases, show that overflow happens exactly when the carry into the sign position differs from the carry out of it, give the two 4-bit examples, conclude $V = C_n \oplus C_{n+1}$ ($C_8 \oplus C_9$ on slide 21).

(b) Signed (2's complement) A, B:



Unsigned version: A < B = C', A = B = Z, A > B = C·Z'. (Gates: one XOR, two NOT, one AND for the signed version.)

(c) Three modules: M1(A, B), M2(C, D), M3(X, 0000).

```
M1: A − B →  A >= B   = (S1 ⊕ V1)'        (signed; unsigned: C1)
M2: C − D →  C != D   = Z2'
M3: X − 0 →  X == 0   = Z3

S1 ─┐
V1 ─┴─[XNOR]─┐
Z2 ──[NOT]───┴─[AND]─┐
Z3 ──────────────────┴─[OR]──► F = (S1 ⊙ V1)·Z2' + Z3
```

Gate count: one XNOR, one NOT, one AND, one OR plus three modules. (If the numbers are unsigned replace the XNOR by C1.)

Quick Notes — ALU Design

Revision points for Lessons 5–9.

L5 · Adders

- HA: S = x ⊕ y, C = xy. FA: S = x ⊕ y ⊕ z, C = xy + xz + yz. Parallel adder = FAs in cascade.
- ALU inputs: A, B, s2 (mode), s1 s0 (function), C_{in}; outputs F, C_{out}.

L6 · Arithmetic circuit

- Y = 0, B, B', 1s ($Y_i = B_i s0 + B_i' s1$); with C_{in} = 0/1 → 8 functions (Table 9-1).
- C_{out} = 1: A + B ≥ 2^n; A + 1 when A = all 1s; A − B − 1 when A > B; A − B when A ≥ B; A − 1 when A ≠ 0.

L7 · X/Y/Z design

- A − B = A + B' + 1; A − B − 1 = A + B'; A − 1 = A + 1s; −A − B − 2 = A' + B'; B − A = A' + B + 1.
- Logic (carries 0): XOR (A, B); XNOR (A, B'); NOT (A, 1); OR (A + B, 0); AND (A + B', B'); AB' (A + B, B); NAND (A' + B', 0).
- $Z_i = s_{mode}' C_i$; C_1 from S bits. Subtractors: F = X − Y − B_{in}, use −B' = B + 1.

L8 · Complete ALU

- Efficient design: arithmetic first; logic from X ⊕ Y; $X_i = A_i + s2s1's0'B_i + s2s1s0'B_i'$, $Y_i = s0B_i + s1B_i'$, $Z_i = s2'C_i$.
- Table 9-4: 8 arithmetic (s2 = 0, Cin) + OR, XOR, AND, complement.

L9 · Status bits

- C = carry out; S = MSB; Z = NOR of all F bits; V = $C_n \oplus C_{n+1}$.
- After A − B: equal Z; signed A < B = S ⊕ V; unsigned A < B = C'.

Key Diagrams to Reproduce

s	A_i	B_i	X_i	Y_i
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	0
1	1	0	1	1
1	1	1	1	0

$X_i = A_i$
$Y_i = B_i \oplus s$
$C_{in} = s$

(c) Truth table and simplified equations

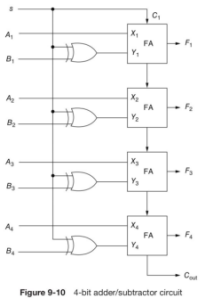


Figure 9-10 4-bit adder/subtractor circuit

Adder/subtractor (Fig. 9-10) — same as Fig. 7.1

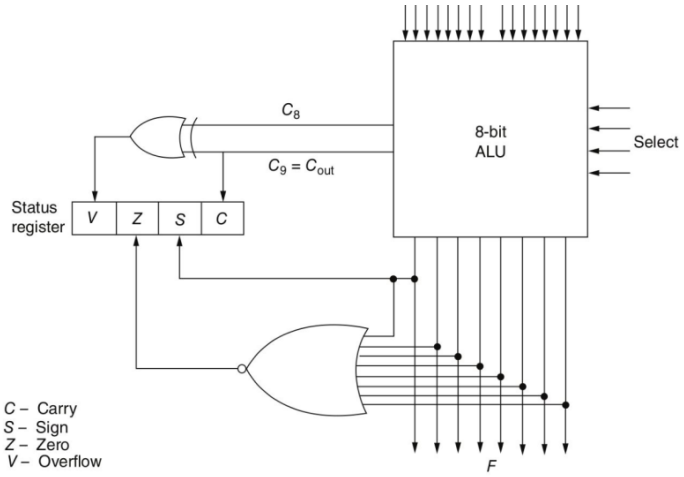


Figure 9-14 Setting bits in a status register

Status register (Fig. 9-14) — same as Fig. 9.1